

Agent Runtime 技术全景

Agent Runtime 可观测性与评估：技术调研报告

覆盖 10 个 observability 子领域项目 | 2026-07-07

1. 背景：Agent 为什么需要专门的观测体系

1.1 核心问题

传统软件的可观测性（Logs / Metrics / Traces 三支柱）对 Agent 系统几乎失效。原因有三：

传统软件	Agent 系统	观测挑战
确定性逻辑	概率性输出（相同输入→不同结果）	无法用“预期结果”断言
单一调用链	多步 Tool Use + 多轮对话	一个用户请求产生数十个 LLM Span
代码级错误	“看起来正确但实际错误”的幻觉	没有 exception，只有不可靠的答案

1.2 Agent 可观测性的三个维度

Agent 可观测性

- 追踪（Tracing）← 看到“发生了什么”
 - LLM 调用链 / Tool 调用 / 检索步骤 / Token 成本
- 评估（Evaluation）← 判断“做对了没有”
 - RAG 质量 / 幻觉检测 / Agent 行为正确性 / 安全合规
- 实验管理（Experimentation）← 找到“怎么做得更好”
 - Prompt 版本对比 / 模型 A/B 测试 / 参数调优

1.3 关键问题

- Agent 非确定性行为的回归测试**：如何在 CI/CD 中对概率性系统做质量门禁？
- LLM-as-Judge 的自我参照偏差**：评估用的 LLM 与被评估的 LLM 存在系统性偏差时如何校准？
- 评估指标体系化**：RAG 质量、Agent 行为、安全性——各自有什么指标，如何组成完整体系？
- 从观测到改进的闭环**：Trace 数据如何反馈到 Prompt 优化和模型选型？

2. 技术方向：五大路线

2.1 全景追踪平台（Full-Stack Tracing）

核心思路：像分布式追踪看微服务一样，看 Agent 的每一步 LLM 调用、Tool Use 和检索操作。

项目	定位	差异化
Langfuse	最流行开源平台，Trace+Eval+Prompt Management 三位一体	ClickHouse 高基数存储，社区最大
Phoenix (Arize)	OpenInference 语义约定，30+ 自动插桩器	Session 聚合 + MCP Server + PXI AI 分析
Opik (Comet)	从 ML 实验到 LLM 追踪的全栈连续性	Comet 老用户继承，Python+Java+TS 全栈
TruLens (Snowflake)	OTel 原生 + RAG 三合一 + Agent 七维评估	反馈函数 DSL，MIT 协议最宽松
OpenLIT	GPU + LLM 全栈	填补 GPU 利用率/VRAM 监控缺口

共同趋势：全部拥抱 **OpenTelemetry** 标准——Phoenix 定义 OpenInference 语义约定，OpenLLMetry 定义 LLM Span 类型，TruLens v2 将其追踪后端替换为 OTel。OTel 正在成为 LLM 可观测性的统一数据总线。

代码规模对比（来自 codebase-memory 架构数据）：

项目	节点数	边数	主要语言
Phoenix	38,303	152,082	TypeScript (2,128) + Python (991)
Opik	92,902	391,722	TypeScript (4,670) + Python (2,498) + Java (1,609)
RAGAS	4,257	19,233	Python (316)

2.2 专项评估框架（Evaluation Frameworks）

核心思路：将 LLM 质量评估包装为可编程测试，融入 CI/CD 质量门禁。

项目	定位	评估对象	核心指数
DeepEval	pytest 式通用评估	RAG / Agent / 幻觉 / 偏见	15+
RAGAS	RAG 专项评估	Retriever + Generator	10+ (RAG 专用)
LM Evaluation Harness	模型基准评测	模型本身（非应用）	200+ 标准化任务

关键洞察： - **DeepEval vs RAGAS**：前者是”评估框架的瑞士军刀”（覆盖面广），后者是”RAG 评估的手术刀”（深度专精）。DeepEval 的指标包括 Answer Relevancy / Faithfulness / Hallucination / Toxicity / Bias；RAGAS 的指标包括 Contextual Precision / Contextual Recall / Faithfulness / Answer Relevancy / Factual Correctness。 - **LM Evaluation Harness** 评估的是**模型能力**（MMLU / GSM8K / HellaSwag），而非**应用质量**。但它是 HuggingFace Open LLM Leaderboard 的核心引擎，直接影响模型选型决策。 - **RAGAS v2 新增**：从 RAG 评估扩展到 Agent 评估，包括 Tool Call Accuracy / Tool Call F1 / Goal Accuracy / Topic Adherence 等指标（从 codebase-memory 热点可见 ascore 为核心入口，fan_in=98）。

2.3 安全评估（Red Teaming）

核心思路：用声明式测试用例系统化检测 Prompt 注入、越狱和敏感信息泄露。

Promptfoo 是这个方向的唯一代表：

YAML 声明式测试 → 50+ LLM Provider 同时执行 → 自动漏洞报告 → CI/CD 安全门禁

- 内置 100+ 注入攻击和越狱 Prompt 模板
- 与 protectai/llm-guard（运行时安全防护）互补：Promptfoo 做评估期测试，LLM-Guard 做生产期防护

2.4 可观测性 SDK / 标准化

OpenLLMetry（Traceloop）的价值不在平台，而在**标准化**：

Agent 应用
↓（OpenLLMetry 自动插件）
OTel Span (llm_request / llm_response / tool_call / retrieval)
↓（OTLP 协议）
任意 OTel 后端（Jaeger / Datadog / Langfuse / Phoenix / Grafana Tempo）

核心矛盾：Phoenix 的 OpenInference 和 OpenLLMetry 都在 OpenTelemetry 之上定义 LLM 专用语义约定，但两者的 Span Kind 和属性命名尚未统一。这是社区标准化的关键博弈点。

2.5 技术路线演进趋势

Phase 1: 专有 SDK 各自为政
Langfuse SDK / Phoenix SDK / TruLens SDK / OpenLLMetry SDK
↓
Phase 2: OpenTelemetry 标准化（当前阶段）
所有平台都支持 OTLP 协议，但语义约定分裂
↓
Phase 3（预测）：统一 LLM OTel 语义约定
OpenInference vs OpenLLMetry 合并或一方胜出
↓
Phase 4（预测）：观测→评估→改进→自动优化闭环
Agent 从 Trace 中自动学习，自我改进 Prompt 和策略

3. 评估指标体系全景

3.1 RAG 评估指标

维度	指标	来源	说明
检索质量	Contextual Precision	RAGAS	检索到的相关文档在结果列表中的排名

维度	指标	来源	说明
检索质量	Contextual Recall	RAGAS	Ground truth 中提到的文档是否被检索到
检索质量	Context Relevance	TruLens	检索文档是否与 query 相关
生成忠实度	Faithfulness	RAGAS / DeepEval	生成的回答中是否有不可从上下文推导的声明
生成忠实度	Groundedness	TruLens	回答是否有上下文支撑
生成质量	Answer Relevancy	RAGAS / DeepEval	回答是否直接回应问题
生成质量	Answer Correctness	RAGAS	事实正确性
幻觉检测	Hallucination	DeepEval	与检索到的上下文对比判断幻觉

3.2 Agent 行为评估指标

维度	指标	来源	说明
规划质量	PlanQuality	TruLens	Agent 制定的计划是否合理
规划遵从	PlanAdherence	TruLens	Agent 是否按计划执行
工具选择	ToolSelection	TruLens	选择的工具是否合适
工具调用	ToolCalling	TruLens	工具调用是否正确
工具质量	ToolQuality	TruLens	工具输出是否满足需求
工具准确性	Tool Call Accuracy	RAGAS v2	实际调用的工具与预期是否一致
逻辑一致性	LogicalConsistency	TruLens	Agent 推理过程是否自治
执行效率	ExecutionEfficiency	TruLens	完成任务所需的步骤数/token 数是否合理
目标达成	Goal Accuracy	RAGAS v2	任务目标是否达成

3.3 安全评估指标

维度	指标	来源	说明
注入防护	Prompt Injection 检测	Promptfoo	是否被恶意 Prompt 绕过
越狱防护	Jailbreak 检测	Promptfoo	是否输出受限内容
毒性	Toxicity	DeepEval	输出是否包含有害内容
偏见	Bias	DeepEval	输出是否存在系统性偏见
敏感信息	PII 泄露	Promptfoo	是否诱导输出敏感信息

3.4 模型能力基准

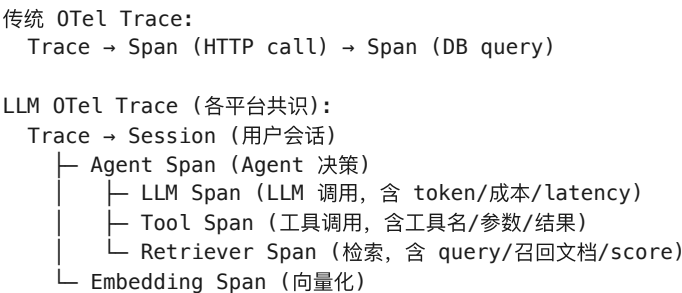
LM Evaluation Harness 覆盖 200+ 标准化评测任务，核心类别包括：

类别	代表任务	评测模式
知识问答	MMLU, ARC	多选题 → loglikelihood
数学推理	GSM8K, MATH	自由生成 → generate_until
常识推理	HellaSwag, WinoGrande	多选题 → loglikelihood
真实性	TruthfulQA	自由生成 → generate_until
代码	HumanEval, MBPP	自由生成 → pass@k

4. 核心设计模式

4.1 Trace 模型：从微服务 Span 到 LLM Span

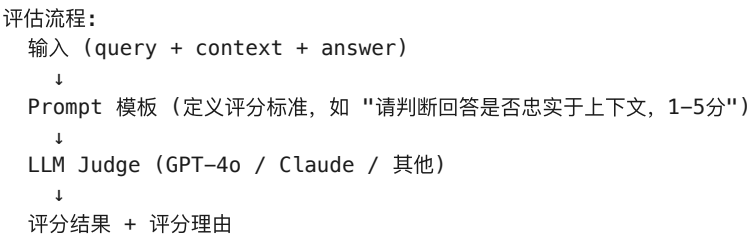
传统分布式追踪的 Span 模型需要为 LLM 场景扩展：



各平台的具体实现： - **Langfuse**: Trace → Generation/Span/Event 三层，Generation 专用于 LLM 调用 - **Phoenix**: OpenInference 定义 LLM/TOOL/AGENT/CHAIN/RETRIEVER/EMBEDDING/RERANKER/GUARDRAIL 等 Span Kind - **Opik**: Trace → Thread → Span，Thread 用于多轮对话分组 - **TruLens**: OTel Span + Lens 选择器，支持 MCP SpanType

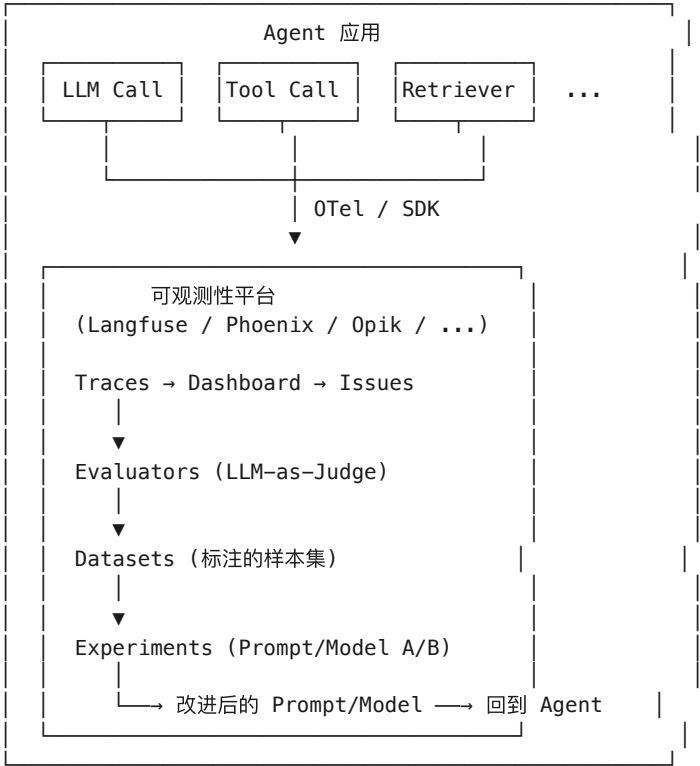
4.2 LLM-as-Judge：自动化评估的核心范式

所有现代 LLM 评估框架都依赖 LLM-as-Judge：



核心挑战（各项目开放问题都指向同一问题）： 1. **Judge Bias**：评估用的 LLM 与被评估的 LLM 存在系统性偏差 2. **Position Bias**： LLM 倾向于给列表靠前的内容更高分 3. **自评偏差**：用同一模型评估自己的输出会高估质量 4. **成本**：每次评估都需要 LLM 调用，token 成本不可忽略

4.3 从观测到改进的闭环



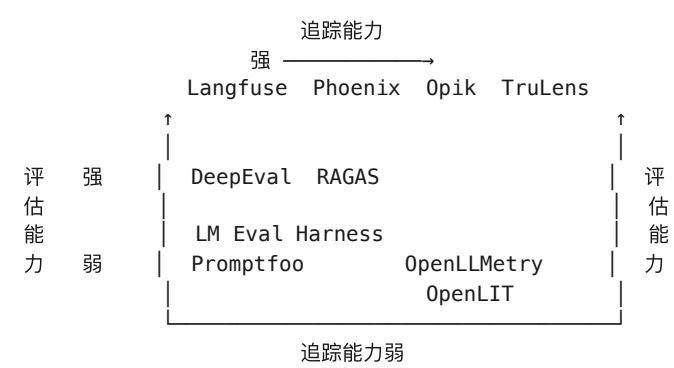
这个闭环（Langfuse 称 “Prompt Engineering Cycle”，Phoenix 称 “Data Flywheel”）是当前所有平台的核心价值主张。

5. 项目全景表

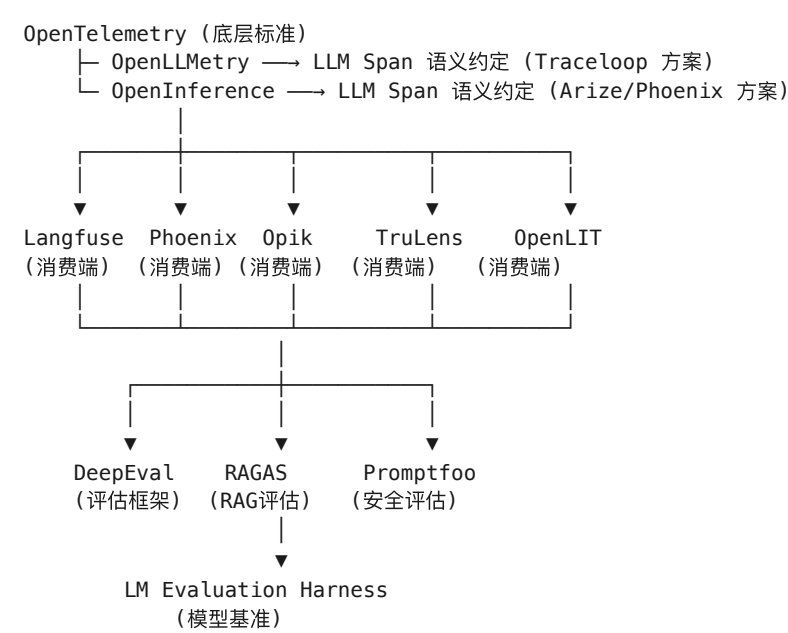
#	项目	Org	GitHub Stars	License	技术栈	核心能力	
1	Langfuse	langfuse	~15K+	MIT+EE	TS/Next.js/ClickHouse/PostgreSQL	Trace/Eval/Prompt Management	追踪+评估+实验
2	Phoenix	Arize AI	~12K+	Elastic 2.0	Python+TS/React/OTLP	OpenInference + 30+ 插件器 + Session + PXI	追踪+评估+实验
3	Opik	Comet ML	~5K+	Apache 2.0	Python+Java+TS/ClickHouse/MySQL	全栈追踪+评估+实验对比	追踪+评估+实验
4	TruLens	Snowflake	~3K+	MIT	Python/OTel/Streamlit	RAG三合一 + Agent 七维评估	追踪+评估
5	DeepEval	Confident AI	~8K+	Apache 2.0	Python/PyTest	pytest式评估 + 15+指标 + Synthesizer	评估
6	RAGAS	VibrantLabsAI	~10K+	Apache 2.0	Python	RAG专项评估 + Agent扩展	评估
7	LM Eval Harness	EleutherAI	~10K+	MIT	Python/PyTorch/HF	200+基准评测 + HF Leaderboard引擎	模型基准
8	Promptfoo	promptfoo	~8K+	MIT	TS/YAML	红队测试 + Prompt Injection	安全评估
9	OpenLLMetry	Traceloop	~4K+	Apache 2.0	Python/OTel	OTel LLM标准化插件SDK	追踪

#	项目	Org	GitHub Stars	License	技术栈	核心能力	可观测性维度
							追踪+硬件监控
10	OpenLIT	openlit	~2K+	Apache 2.0	Python/OTel/DCGM	GPU+LLM全栈监控	

5.1 按能力维度分类



5.2 生态系统关系图



6. 趋势与展望

6.1 2025-2026 关键趋势

- 1. **OpenTelemetry 一统天下**：所有主流平台都已完成或正在完成 OTel 迁移。Phoenix/OpenLLMetry/OpenLIT/TruLens v2 全部基于 OTLP。Langfuse 虽有自己的 SDK，但也支持 OTel 导入。未来的竞争不在数据采集层，而在**语义约定标准化**和**消费体验**。
- 2. **从评估到自动优化**：RAGAS v2 新增 optimizer 模块（DSPy/遗传算法），DeepEval 有 Synthesizer 自动生成评估数据——评估框架正在从”告诉你哪里不好”扩展到”帮你变得更好”。
- 3. **Agent 专项评估成为新战场**：RAGAS 从纯 RAG 扩展到 Tool Call Accuracy / Goal Accuracy，TruLens 推出七维 Agent 评估器——Agent 行为的可测量性是 2026 年的核心命题。
- 4. **安全评估从可选变为必需**：Promptfoo 代表的红队测试正在成为 CI/CD 安全门禁的标准组件，与 OWASP LLM Top 10 对齐。
- 5. **成本可观测性**：Phoenix 和 Langfuse 都将 token 成本追踪作为核心功能——Agent 调用链的成本归因直接影响生产部署决策。

6.2 未解决的关键问题

问题	影响范围	当前状态
LLM-as-Judge 偏差校准	所有评估框架	无统一方案，各框架自行设计
OTel LLM Span 语义标准化	Phoenix vs OpenLLMetry	分裂中，上游 OTel 尚未采纳
Agent 非确定性回归测试	CI/CD 集成	各框架提供采样/统计方案，无确定性保证
评估数据的 ground truth 生成	RAGAS / DeepEval	Synthesizer/TestsetGen 自动生成，质量待验证
开源版 vs 企业版功能边界	Langfuse / Phoenix (ELv2)	Langfuse MIT+EE, Phoenix ELv2 商业限制
多模态 Agent 评估	RAGAS / TruLens	RAGAS 有 multi_modal_faithfulness/relevance，刚起步

6.3 推荐关注方向

对 Agent 团队而言，可观测性技术栈建议如下：

- 必选层：
 - └ 追踪平台：Langfuse（社区最大）或 Phoenix（OTel 标准最完善）
 - └ 评估框架：DeepEval（通用）+ RAGAS（RAG 专项）
- 安全层：
 - └ 红队测试：Promptfoo → CI/CD 安全门禁
- 模型选型：
 - └ 基准评测：LM Evaluation Harness → 参考 HF Leaderboard
- 可选层：

- └ GPU 监控: OpenLIT (自建推理时需要)
- └ OTel 标准化: OpenLLMetry (需要与现有 OTel 栈集成时)

参考资料

- [Langfuse](#) — 开源 LLM 可观测性平台
- [Arize Phoenix](#) — OpenInference 可观测性平台
- [Comet Opik](#) — 全栈 LLM 追踪与评估
- [TruLens](#) — Snowflake 评估与可观测性框架
- [DeepEval](#) — pytest 式 LLM 评估
- [RAGAS](#) — RAG 评估框架
- [LM Evaluation Harness](#) — 模型基准评测
- [Promptfoo](#) — LLM 红队测试
- [OpenLLMetry](#) — OTel LLM 可观测性 SDK
- [OpenLIT](#) — GPU+LLM 全栈可观测
- [OpenTelemetry](#) — 可观测性标准
- [HuggingFace Open LLM Leaderboard](#) — 模型能力排行

本报告由 KG System 基于 10 个 observability 子领域项目生成 | 2026-07-07